

NetCon v1 By CTTeddy Documentation



github.com/CTTeddy/NetConV1

github.com/CTTeddy/NetConV1/blob/main/NetConDocumentation.pdf

1. Purpose of the Project

NetCon v1 is a centralized, single-channel networking and communication architecture built for Roblox experiences. Its primary goal is to eliminate the chaos of managing dozens of scattered RemoteEvents and RemoteFunctions across ReplicatedStorage.

By consolidating all network traffic through a single secure main channel (NetConMainChannel), it drastically reduces network overhead, hard-codes server-authoritative security, and integrates built-in rate-limiting/anti-spam protection to prevent client-side exploitation.

2. Difference Between NetCon v1 and Standard RemoteEvents

Feature	Standard RemoteEvents	NetCon v1
Workspace Organization	Clutters ReplicatedStorage with dozens of separate remote instances for every action.	Keeps ReplicatedStorage clean using a single dedicated folder structure and one master remote.
Security & Validation	Requires writing custom rate-limiting, spam checks, and argument verification for every single remote.	Built-in automatic anti-spam throttling (blocks excessive requests) and type-checking out of the box.
Maintainability	Hard to track; if a remote name changes or gets exploited, it breaks or creates severe security vulnerabilities.	Centralized routing via Register and Fire handles actions seamlessly through string identifiers.

3. Difference Between NetCon and Knit

- **Architectural Scope:** Knit is a massive, full-scale framework designed to structure an entire Roblox game using Services, Controllers, and component lifecycles. **NetCon**, on the other hand, is laser-focused *strictly* on networking and communication security.
- **Complexity & Boilerplate:** Knit requires setting up service/controller loaders, boilerplate structures, and specific component dependencies. NetCon requires zero complex framework setup—you simply require the module, run `Init()`, and immediately start registering actions.
- **Learning Curve:** While Knit can feel overwhelming or bloated for smaller projects or developers who just want to fix exploits quickly, NetCon is lightweight and acts as a drop-in networking layer.

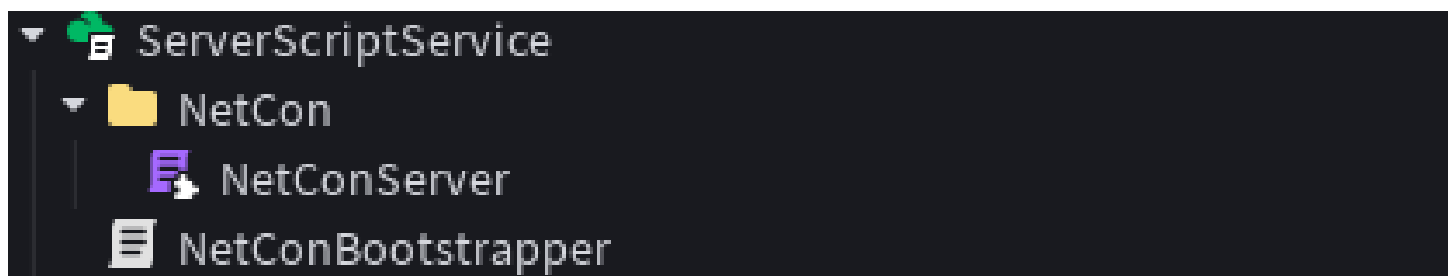
4. Is It Beginner-Friendly?

Yes, absolutely. NetCon v1 is designed with beginners in mind for several reasons:

- **Plug-and-Play Setup:** You only need two lines of initialization code (one for the server, one for the client).
- **No Complex Folders:** You don't have to worry about instantiating and placing multiple remotes manually; the system initializes its own channels safely.
- **Intuitive API:** Using `.Register()` on the server and `.Fire()` / `.On()` on the client mimics standard event-driven programming, making it extremely easy to understand and adopt without getting lost in advanced software design patterns.

5. Usage

--Server Side Init--



```
local NetConServer = require(script.Parent.NetCon.NetConServer)
NetConServer.Init()
```

--Client Side Init--

StarterPlayerScripts

NetConClient

```
local NetCon = require(game:GetService("ReplicatedStorage").NetCon.Shared.NetCon)
NetCon.Init()
```

6. API Reference & Code Examples

Example Code For Server:

```
-- File: ServerScriptService/TestScriptNetCon
local ServerScriptService = game:GetService("ServerScriptService")
local NetConServer = require(ServerScriptService:WaitForChild("NetCon"):WaitForChild("NetConServer"))

NetConServer.Register("RequestPowerUpgrade", function(player)
    -- 1. Server-Authoritative Logic: Never trust the client's data.
    -- Instead, fetch the player's actual level from data stores or leaderstats.
    -- local currentLevel = player.leaderstats.Power.Value

    -- 2. Calculate the next upgrade step safely on the server side
    -- local nextLevel = currentLevel + 1

    -- 3. Validate if the player has enough currency/requirements before proceeding

    print(player.Name .. " upgraded their power successfully.")

    NetConServer.FireClient(player, "PowerResult", "Power successfully upgraded!")
end)
```

Example Code For Client:

```
-- File: StarterPlayerScripts/TestLocalScriptNetCon
local ReplicatedStorage = game:GetService("ReplicatedStorage")
local UserInputService = game:GetService("UserInputService")
local NetCon = require(ReplicatedStorage:WaitForChild("NetCon"):WaitForChild("Shared"):WaitForChild("NetCon"))

local function RequestPowerUpgrade()
    -- Client only fires the request without passing manipulable parameters
    NetCon.Fire("RequestPowerUpgrade")
end

NetCon.On("PowerResult", function(message)
    -- Handle the server response (e.g., update UI)
end)

UserInputService.InputBegan:Connect(function(input, gameProcessed)
    if gameProcessed then return end

    if input.KeyCode == Enum.KeyCode.K then
        RequestPowerUpgrade()
    end
end)
```

“

"Never Trust the Client!"

— CTTeddy and the Most Pro Devs

”



Strict Terms of Use & Copyright Notice

By accessing, downloading, integrating, or utilizing NetCon v1, you agree to the following legally binding terms:

All Rights Reserved: Exclusive intellectual property of the developer.

Absolute Prohibition of Modification: Altering any portion of the core source code is strictly prohibited.

Mandatory Preservation of Verification Warnings: Startup verification warnings must remain intact.

Strict Anti-Redistribution & Anti-Monetization Policy: Redistribution, resale, or unauthorized distribution is forbidden.

Enforcement & Legal Action: Unauthorized use will result in DMCA takedowns and formal legal action.



Credits & Metadata

Developer: CTTeddy

Project: NetCon v1 Simple Network & Communication System